

CHAPTER 18

Introduction to Scikit-learn

Scikit-learn is a Python library that provides a standard interface for implementing machine learning algorithms. It includes other ancillary functions that are integral to the machine learning pipeline such as data preprocessing steps, data resampling techniques, evaluation parameters, and search interfaces for tuning/optimizing an algorithm's performance.

This section will go through the functions for implementing a typical machine learning pipeline with Scikit-learn. Since, Scikit-learn has a variety of packages and modules that are called depending on the use case, we'll import a module directly from a package if and when needed using the **from** keyword. Again the goal of this material is to provide the foundation to be able to comb through the exhaustive Scikit-learn library and be able to use the right tool or function to get the job done.

Loading Sample Datasets from Scikit-learn

Scikit-learn comes with a set of small standard datasets for quickly testing and prototyping machine learning models. These datasets are ideal for learning purposes when starting off working with machine learning or even trying out the performance of some new model. They save a bit of the time required to identify, download, and clean up a dataset obtained from the wild. However, these datasets are small and well curated, they do not represent real-world scenarios.

Five popular sample datasets are

- Boston house-prices dataset
- Diabetes dataset

- Iris dataset
- Wisconsin breast cancer dataset
- Wine dataset

Table 18-1 summarizes the properties of these datasets.

Table 18-1. *Scikit-learn Sample Dataset Properties*

Dataset name	Observations	Dimensions	Features	Targets
Boston house-prices dataset (regression)	506	13	real, positive	real 5.–50.
Diabetes dataset (regression)	442	10	real, $-0.2 < x < 0.2$	integer 25–346
Iris dataset (classification)	150	4	real, positive	3 classes
Wisconsin breast cancer dataset (classification)	569	30	real, positive	2 classes
Wine dataset (classification)	178	13	real, positive	3 classes

To load the sample dataset, we'll run

```
# load library
from sklearn import datasets
import numpy as np
```

Load the Iris dataset

```
# load iris
iris = datasets.load_iris()
iris.data.shape
'Output': (150, 4)
iris.feature_names
'Output':
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

Methods for loading other datasets:

- Boston house-prices dataset – **datasets.load_boston()**
- Diabetes dataset – **datasets.load_diabetes()**
- Wisconsin breast cancer dataset – **datasets.load_breast_cancer()**
- Wine dataset – **datasets.load_wine()**

Splitting the Dataset into Training and Test Sets

A core practice in machine learning is to split the dataset into different partitions for training and testing. Scikit-learn has a convenient method to assist in that process called **train_test_split(X, y, test_size=0.25)**, where **X** is the design matrix or dataset of predictors and **y** is the target variable. The split size is controlled using the attribute **test_size**. By default, **test_size** is set to 25% of the dataset size. It is standard practice to shuffle the dataset before splitting by setting the attribute **shuffle=True**.

```
# import module
from sklearn.model_selection import train_test_split
# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
shuffle=True)

X_train.shape
'Output': (112, 4)
X_test.shape
'Output': (38, 4)
y_train.shape
'Output': (112,)
y_test.shape
'Output': (38,)
```

Preprocessing the Data for Model Fitting

Before a dataset is trained or fitted with a machine learning model, it necessarily undergoes some vital transformations. These transformations have a huge effect on the performance of the learning model. Transformations in Scikit-learn have a **fit()** and **transform()** method, or a **fit_transform()** method.

Depending on the use case, the **fit()** method can be used to learn the parameters of the dataset, while the **transform()** method applies the data transform based on the learned parameters to the same dataset and also to the test or validation datasets before modeling. Also, the **fit_transform()** method can be used to learn and apply the transformation to the same dataset in a one-off fashion. Data transformation packages are found in the **sklearn.preprocessing** package.

This section will cover some critical transformation for numeric and categorical variables. They include

- Data rescaling
- Standardization
- Normalization
- Binarization
- Encoding categorical variables
- Input missing data
- Generating higher-order polynomial features

Data Rescaling

It is often the case that the features of the dataset contain data with different scales. In other words, the data in column A can be in the range of 1–5, while the data in column B is in the range of 1000–9000. This different scale for units of observations in the same dataset can have an adverse effect for certain machine learning models, especially when minimizing the cost function of the algorithm because it shrinks the function space and makes it difficult for an optimization algorithm like gradient descent to find the global minimum.

When performing data rescaling, usually the attributes are rescaled with the range of 0 and 1. Data rescaling is implemented in Scikit-learn using the **MinMaxScaler** module. Let's see an example.

```
# import packages
from sklearn import datasets
from sklearn.preprocessing import MinMaxScaler
```

```

# load dataset
data = datasets.load_iris()
# separate features and target
X = data.data
y = data.target

# print first 5 rows of X before rescaling
X[0:5,:]
'Output':
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])

# rescale X
scaler = MinMaxScaler(feature_range=(0, 1))
rescaled_X = scaler.fit_transform(X)

# print first 5 rows of X after rescaling
rescaled_X[0:5,:]
'Output':
array([[0.22222222, 0.625      , 0.06779661, 0.04166667],
       [0.16666667, 0.41666667, 0.06779661, 0.04166667],
       [0.11111111, 0.5        , 0.05084746, 0.04166667],
       [0.08333333, 0.45833333, 0.08474576, 0.04166667],
       [0.19444444, 0.66666667, 0.06779661, 0.04166667]])

```

Standardization

Linear machine learning algorithms such as linear regression and logistic regression make an assumption that the observations of the dataset are normally distributed with a mean of 0 and standard deviation of 1. However, this is often not the case with real-world datasets as features are often skewed with differing means and standard deviations.

Applying the technique of standardization to the datasets transforms the features into a standard Gaussian (or normal) distribution with a mean of 0 and standard deviation of 1. Scikit-learn implements data standardization in the **StandardScaler** module. Let's look at an example.

```
# import packages
from sklearn import datasets
from sklearn.preprocessing import StandardScaler

# load dataset
data = datasets.load_iris()
# separate features and target
X = data.data
y = data.target

# print first 5 rows of X before standardization
X[0:5,:]
'Output':
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])

# standardize X
scaler = StandardScaler().fit(X)
standardize_X = scaler.transform(X)

# print first 5 rows of X after standardization
standardize_X[0:5,:]
'Output':
array([[ -0.90068117,  1.03205722, -1.3412724 , -1.31297673],
       [-1.14301691, -0.1249576 , -1.3412724 , -1.31297673],
       [-1.38535265,  0.33784833, -1.39813811, -1.31297673],
       [-1.50652052,  0.10644536, -1.2844067 , -1.31297673],
       [-1.02184904,  1.26346019, -1.3412724 , -1.31297673]])
```

Normalization

Data normalization involves transforming the observations in the dataset so that it has a unit norm or has magnitude or length of 1. The length of a vector is the square root of the sum of squares of the vector elements. A unit vector (or unit norm) is obtained by dividing the vector by its length. Normalizing the dataset is particularly useful in scenarios where the dataset is sparse (i.e., a large number of observations are zeros) and also has differing scales. Normalization in Scikit-learn is implemented in the **Normalizer** module.

```
# import packages
from sklearn import datasets
from sklearn.preprocessing import Normalizer

# load dataset
data = datasets.load_iris()
# separate features and target
X = data.data
y = data.target

# print first 5 rows of X before normalization
X[0:5,:]
'Output':
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])

# normalize X
scaler = Normalizer().fit(X)
normalize_X = scaler.transform(X)

# print first 5 rows of X after normalization
normalize_X[0:5,:]
'Output':
```

```
array([[0.80377277, 0.55160877, 0.22064351, 0.0315205 ],
       [0.82813287, 0.50702013, 0.23660939, 0.03380134],
       [0.80533308, 0.54831188, 0.2227517 , 0.03426949],
       [0.80003025, 0.53915082, 0.26087943, 0.03478392],
       [0.790965 , 0.5694948 , 0.2214702 , 0.0316386 ]])
```

Binarization

Binarization is a transformation technique for converting a dataset into binary values by setting a cutoff or threshold. All values above the threshold are set to 1, while those below are set to 0. This technique is useful for converting a dataset of probabilities into integer values or in transforming a feature to reflect some categorization. Scikit-learn implements binarization with the **Binarizer** module.

```
# import packages
from sklearn import datasets
from sklearn.preprocessing import Binarizer

# load dataset
data = datasets.load_iris()
# separate features and target
X = data.data
y = data.target

# print first 5 rows of X before binarization
X[0:5,:]
'Output':
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])

# binarize X
scaler = Binarizer(threshold = 1.5).fit(X)
binarize_X = scaler.transform(X)
```



```
# print first 5 rows of X after binarization
binarize_X[0:5,:]
'Output':
array([[1., 1., 0., 0.],
       [1., 1., 0., 0.],
       [1., 1., 0., 0.],
       [1., 1., 0., 0.],
       [1., 1., 0., 0.]])
```

Encoding Categorical Variables

Most machine learning algorithms do not compute with non-numerical or categorical variables. Hence, encoding categorical variables is the technique for converting non-numerical features with labels into a numerical representation for use in machine learning modeling. Scikit-learn provides modules for encoding categorical variables including the **LabelEncoder** for encoding labels as integers, **OneHotEncoder** for converting categorical features into a matrix of integers, and **LabelBinarizer** for creating a one-hot encoding of target labels.

LabelEncoder is typically used on the target variable to transform a vector of hashable categories (or labels) into an integer representation by encoding label with values between 0 and the number of categories minus 1. This is further illustrated in Figure 18-1.

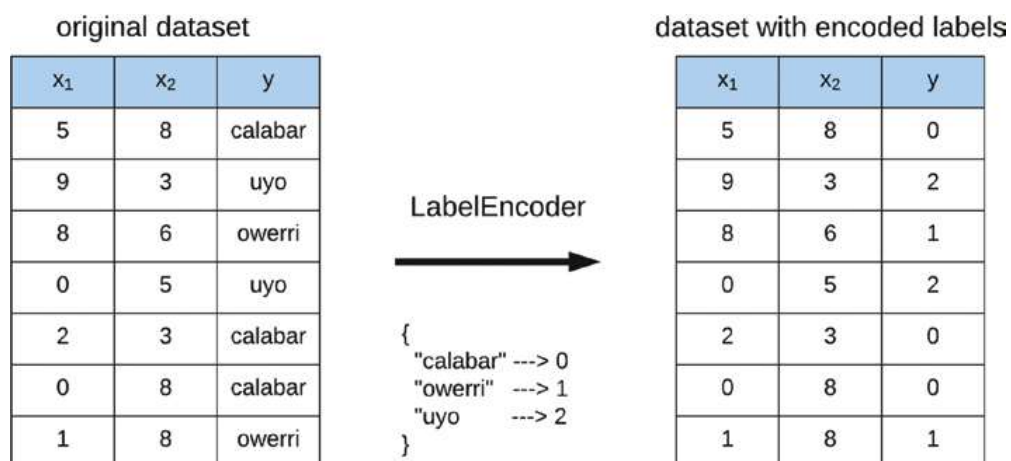


Figure 18-1. *LabelEncoder*

Let's see an example of **LabelEncoder**.

```
# import packages
from sklearn.preprocessing import LabelEncoder

# create dataset
data = np.array([[5,8,"calabar"],[9,3,"uyo"],[8,6,"owerri"],
                 [0,5,"uyo"],[2,3,"calabar"],[0,8,"calabar"],
                 [1,8,"owerri"]])

data
'Output':
array([[ '5', '8', 'calabar'],
       [ '9', '3', 'uyo'],
       [ '8', '6', 'owerri'],
       [ '0', '5', 'uyo'],
       [ '2', '3', 'calabar'],
       [ '0', '8', 'calabar'],
       [ '1', '8', 'owerri']], dtype='<U21')

# separate features and target
X = data[:, :2]
y = data[:, -1]

# encode y
encoder = LabelEncoder()
encode_y = encoder.fit_transform(y)

# adjust dataset with encoded targets
data[:, -1] = encode_y
data
'Output':
array([[ '5', '8', '0'],
       [ '9', '3', '2'],
       [ '8', '6', '1'],
       [ '0', '5', '2'],
       [ '2', '3', '0'],
       [ '0', '8', '0'],
       [ '1', '8', '1']], dtype='<U21')
```

OneHotEncoder is used to transform a categorical feature variable in a matrix of integers. This matrix is a sparse matrix with each column corresponding to one possible value of a category. This is further illustrated in Figure 18-2.

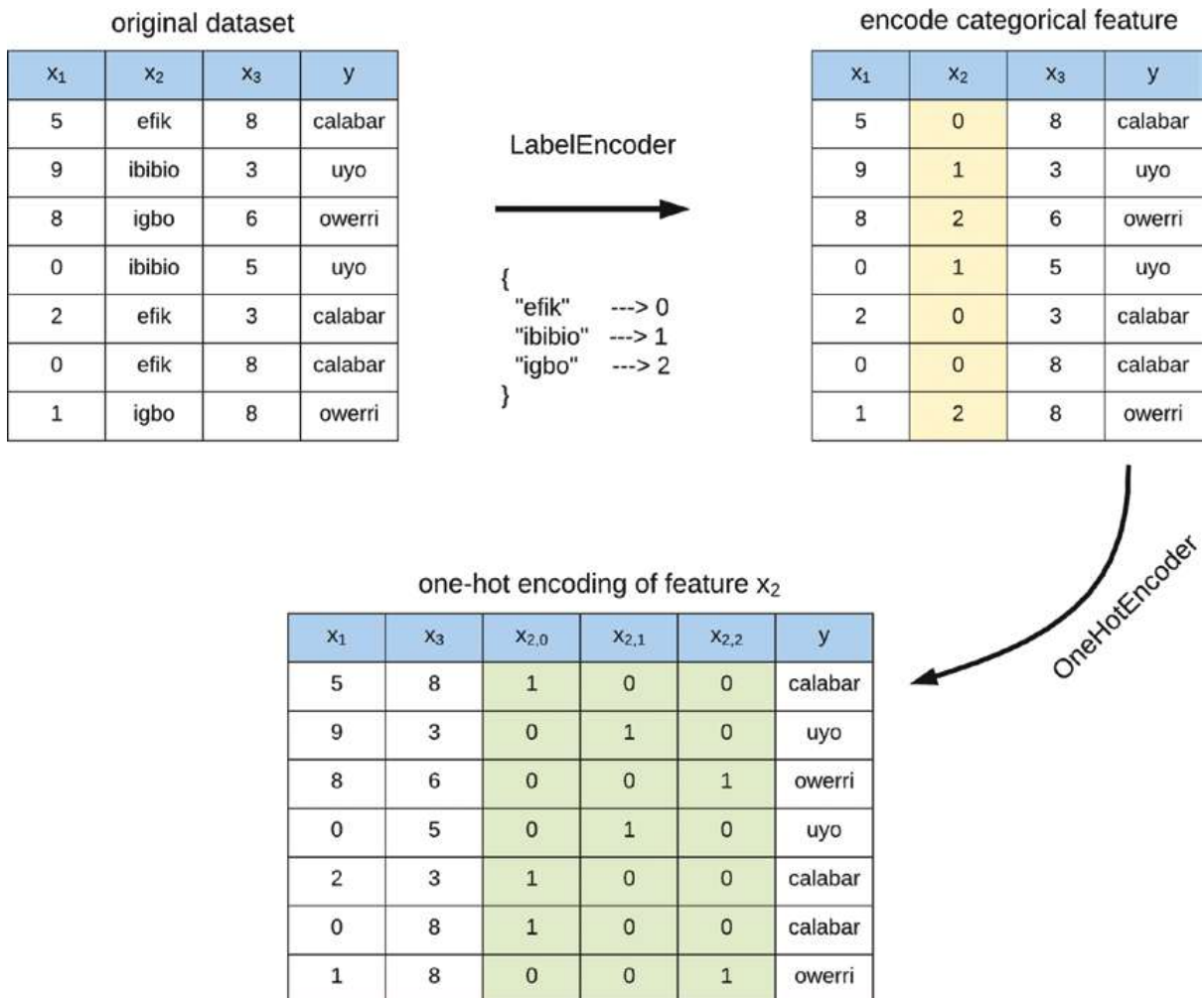


Figure 18-2. *OneHotEncoder*

Let's see an example of **OneHotEncoder**.

```
# import packages
from sklearn.preprocessing import OneHotEncoder
```

```

# create dataset
data = np.array([[5,"efik", 8,"calabar"],[9,"ibibio",3,"uyo"],[8,"igbo",
6,"owerri"],[0,"ibibio",5,"uyo"],[2,"efik",3,"calabar"],[0,"efik",
8,"calabar"],[1,"igbo",8,"owerri"]])

# separate features and target
X = data[:, :3]
y = data[:, -1]

# print the feature or design matrix X
X
'Output':
array([[ '5', 'efik', '8'],
       [ '9', 'ibibio', '3'],
       [ '8', 'igbo', '6'],
       [ '0', 'ibibio', '5'],
       [ '2', 'efik', '3'],
       [ '0', 'efik', '8'],
       [ '1', 'igbo', '8']], dtype='<U21')

# one_hot_encode X
one_hot_encoder = OneHotEncoder(handle_unknown='ignore')
encode_categorical = X[:,1].reshape(len(X[:,1]), 1)
one_hot_encode_X = one_hot_encoder.fit_transform(encode_categorical)

# print one_hot encoded matrix - use todense() to print sparse matrix
# or convert to array with toarray()
one_hot_encode_X.todense()
'Output':
matrix([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.],
        [0., 1., 0.],
        [1., 0., 0.],
        [1., 0., 0.],
        [0., 0., 1.]])

```

```
# remove categorical label
X = np.delete(X, 1, axis=1)
# append encoded matrix
X = np.append(X, one_hot_encode_X.toarray(), axis=1)
X
'Output':
array([[ '5', '8', '1.0', '0.0', '0.0'],
       [ '9', '3', '0.0', '1.0', '0.0'],
       [ '8', '6', '0.0', '0.0', '1.0'],
       [ '0', '5', '0.0', '1.0', '0.0'],
       [ '2', '3', '1.0', '0.0', '0.0'],
       [ '0', '8', '1.0', '0.0', '0.0'],
       [ '1', '8', '0.0', '0.0', '1.0']], dtype='<U32')
```

Input Missing Data

It is often the case that a dataset contains several missing observations. Scikit-learn implements the **Imputer** module for completing missing values.

```
# import packages
from sklearn. impute import SimpleImputer

# create dataset
data = np.array([[5,np.nan,8],[9,3,5],[8,6,4],
                 [np.nan,5,2],[2,3,9],[np.nan,8,7],
                 [1,np.nan,5]])

data
'Output':
array([[ 5., nan,  8.],
       [ 9.,  3.,  5.],
       [ 8.,  6.,  4.],
       [nan,  5.,  2.],
       [ 2.,  3.,  9.],
       [nan,  8.,  7.],
       [ 1., nan,  5.]])
```

```
# impute missing values - axis=0: impute along columns
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
imputer.fit_transform(data)
'Output':
array([[5., 5., 8.],
       [9., 3., 5.],
       [8., 6., 4.],
       [5., 5., 2.],
       [2., 3., 9.],
       [5., 8., 7.],
       [1., 5., 5.]])
```

Generating Higher-Order Polynomial Features

Scikit-learn has a module called `PolynomialFeatures` for generating a new dataset containing high-order polynomial and interaction features based off the features in the original dataset. For example, if the original dataset has two dimensions $[a, b]$, the second-degree polynomial transformation of the features will result in $[1, a, b, a^2, ab, b^2]$.

```
# import packages
from sklearn.preprocessing import PolynomialFeatures

# create dataset
data = np.array([[5,8],[9,3],[8,6],
                 [5,2],[3,9],[8,7],
                 [1,5]])

data
'Output':
array([[5, 8],
       [9, 3],
       [8, 6],
       [5, 2],
       [3, 9],
       [8, 7],
       [1, 5]])
```

```
# create polynomial features
polynomial_features = PolynomialFeatures(2)
data = polynomial_features.fit_transform(data)
data
'Output':
array([[ 1.,  5.,  8., 25., 40., 64.],
       [ 1.,  9.,  3., 81., 27.,  9.],
       [ 1.,  8.,  6., 64., 48., 36.],
       [ 1.,  5.,  2., 25., 10.,  4.],
       [ 1.,  3.,  9.,  9., 27., 81.],
       [ 1.,  8.,  7., 64., 56., 49.],
       [ 1.,  1.,  5.,  1.,  5., 25.]])
```

Machine Learning Algorithms

This chapter provides an introduction to working with the Scikit-learn library for implementing machine learning algorithms.

In the next chapters, we'll implement supervised and unsupervised machine learning models using Scikit-learn. Scikit-learn provides a consistent set of methods, which are the **fit()** method for fitting models to the training dataset and the **predict()** method for using the fitted parameters to make a prediction on the test dataset. The examples are geared at explaining working with Scikit-learn; hence, we are not so keen on the performance of the model.